

Restaurant Order & Management System

Scope: Robust CRUD System, Authentication, Session Management, and Database Schema

1. System Architecture Overview

This technical specification outlines a robust, secure, and scalable Restaurant Order System designed using a traditional Model-View-Controller (MVC) architecture pattern or a clean modular structured approach. The system emphasizes backend security, efficient relational data mapping, and precise computation of commercial transactions.



2. Database Schema & Relational Design

The persistence layer is structured using relational tables ensuring strict cascading rules and data integrity. All monetary fields utilize exact precision data types to avoid standard floating-point inaccuracies.

Table Name	Primary Key	Foreign Keys / Relations	Core Attributes & Types
users	id (INT)	None	username (VARCHAR), email (VARCHAR), password_hash (VARCHAR), role (ENUM: admin, staff, customer)
categories	id (INT)	None	name (VARCHAR), description (TEXT)
menu_items	id (INT)	category_id -> categories(id)	name (VARCHAR), price (DECIMAL(10,2)), status (AVAILABLE, OUT_OF_STOCK)
orders	id (INT)	user_id -> users(id)	order_date (TIMESTAMP), total_amount (DECIMAL(10,2)), status (PENDING, PREPARING, DELIVERED, CANCELLED)
order_items	id (INT)	order_id -> orders(id), menu_item_id	quantity (INT), unit_price (DECIMAL(10,2))

3. Core Functional Modules

3.1 Authentication & Authorization Layer

- Secure User registration and encrypted login using state-of-the-art password hashing algorithms (`password_hash()` with `PASSWORD_BCRYPT`).
- Strict Session-based authorization checking privileges across different administrative scopes (e.g., restricted dashboard access for standard kitchen staff).

3.2 Menu & Category Management (CRUD)

- Complete CRUD lifecycle for menu categories and individual catalog listings.
- Dynamic upload structures with strict MIME-type validation for meal presentation imagery.

3.3 Order Processing & Lifecycle Engine

- Cart management system backed by stateful sessions or highly available database tables.
- Transactional database queries executing safe atomicity controls to protect order integrity during high-concurrency peak operational hours.

4. Technical Stack & Implementation Guidelines

Recommended Environment Layout:

Language: PHP 8.x+

Database Engine: MySQL / MariaDB

Driver: PDO (PHP Data Objects)

Security Layer: Prepared Statements

Implementation Note on Financial Calculations:

The gross calculation of any order total value T across individual items containing quantity q_i and base price p_i must follow strict accumulation loops:

$$T = \sum (q_i \times p_i)$$

Always utilize parametrized bindings within your PDO instances to guarantee immunity against malicious SQL Injection vectors:

```
$stmt = $pdo->prepare("INSERT INTO order_items (order_id, menu_item_id, quantity)
VALUES (:order_id, :item_id, :qty)");
```

5. Recommended Step-by-Step Project Phases

- **Phase 1: DB Blueprinting:** Construct and run the DDL schema files incorporating foreign key constraints.
- **Phase 2: Authentication Engine:** Build connection singletons, register dashboards, handle layout security barriers.
- **Phase 3: Catalog Setup:** Implement forms for category additions and item manipulation operations.

- **Phase 4: Checkout Loop:** Setup session tracking mechanisms to assemble menu items and securely insert into orders pipeline.